Tradurre in linguaggio C la seguente porzione di codice assembler
I simboli VETTA, VETTB, VETTC, K1, e K2 sono costanti.

```
    add t6, zero, zero
    addi t7, zero, 400
L1: beq t6, t7, END
    lw t2, VETTA(t6)
    lw t3, VETTB(t6)
    slt t0, t2, t3
    bne t0, 0, L3
    addi t2, t2, K1
    sw t2, VETTA(t6)
    addi t3, t3, K2
    sw t3, VETTB(t6)
    add t4, t2, t3
    sw t4, VETTC(t6)
    sw zero, VETTD(t6)
    j INC
L3: sw zero, VETTC(t6)
    sub t1, t2, t3
    sw t1, VETTD(t6)
INC: addi t6, t6, 4
    j L1
END:
```

Tradurre in linguaggio C la seguente porzione di codice assembler
I simboli VETTA, VETTB, VETTC, K1, e K2 sono costanti.

add t6, zero, zero	X ← 0;
addi t7, zero, 400	Y ← 100;
L1: beq t6, t7, END	if (X == Y) goto END;
lw t2, VETTA(t6)	var2 ← VETTA[X];
lw t3, VETTB(t6)	var3 ← VETTB[X];
slt t0, t2, t3	t0 ←1 se var2 < var3 ;
bne t0, 0, L3	if t0 != 0 goto L3;
addi t2, t2, K1	var2 ← var2 + K1;
sw t2, VETTA(t6)	VETTA[X] ← var2;
addi t3, t3, K2	var3 ← var3 + K2;
sw t3, VETTB(t6)	VETTB[X] ← var3;
add t4, t2, t3	var4 ← var2 + var3;
sw t4, VETTC(t6)	VETTC[X] ← var4;
sw zero, VETTD(t6)	VETTD[X] ← 0;
j INC	goto INC;
L3: sw zero, VETTC(t6)	VETTC[X] ← 0;
sub t1, t2, t3	var1 ← var2 - var3;
sw t1, VETTD(t6)	VETTD[X] ← var1;
INC: addi t6, t6, 4	i++;
j L1	goto L1;
END:	

Tradurre in linguaggio C la seguente porzione di codice assembler
I simboli VETTA, VETTB, VETTC, K1, e K2 sono costanti.

add t6, zero, zero	X ← 0;	i =0;
addi t7, zero, 400	Y ← 100;	N =100;
L1: beq t6, t7, END	if (X == Y) goto END;	while (i != N) {
lw t2, VETTA(t6)	var2 ← VETTA[X];	
lw t3, VETTB(t6)	var3 ← VETTB[X];	
slt t0, t2, t3	t0 ←1 se var2 < var3 ;	
bne t0, 0, L3	if t0 != 0 goto L3;	
addi t2, t2, K1	var2 ← var2 + K1;	
sw t2, VETTA(t6)	VETTA[X] ← var2;	VETTA[i] = VETTA[i] + K1;
addi t3, t3, K2	var3 ← var3 + K2;	
sw t3, VETTB(t6)	VETTB[X] ← var3;	VETTB[i] = VETTB[i] + K2;
add t4, t2, t3	var4 ← var2 + var3;	
sw t4, VETTC(t6)	VETTC[X] ← var4;	VETTC[i] = VETTA[i] + VETTB[i];
sw zero, VETTD(t6)	VETTD[X] ← 0;	VETTD[i] = 0;
j INC	goto INC;	else
L3: sw zero, VETTC(t6)	VETTC[X] ← 0;	VETTC[i] = 0;
sub t1, t2, t3	var1 ← var2 - var3;	
sw t1, VETTD(t6)	VETTD[X] ← var1;	VETTD[i] = VETTA[i] - VETTB[i]; }
INC: addi t6, t6, 4	i++;	i++;
j L1	goto L1;	}
END:		

```
i =0;
N =100;
while (i != N) {
if (vettA[i] >= vettB[i]) {
        VETTA[i] = VETTA[i] + K1;
        VETTB[i] = VETTB[i] + K2;
        VETTC[i] = VETTA[i] + VETTB[i];
        VETTD[i] = 0;
}
else {
        VETTC[i] = 0;
        VETTD[i] = VETTA[i] - VETTB[i];
}
i++;
}
```